

소프트웨어/인공지능 분과

Virtual Human 에이전트를 활용한 가상 면접 플랫폼 중간보고서



팀: VRoom

202155575 유재명 ryujm3410@pusan.ac.kr

202355503 강저릭 어요솅드 oganzorig12@pusan.ac.kr

202055503 강동요 krgdy@pusan.ac.kr

지도교수: 이명호

1. 요구조건 및 제약 사항 분석에 대한 수정사항

1.1 페르소나 요구조건의 변경

착수보고서에서는 지원 직무에 맞춘 고유한 성격과 전문지식을 갖춘 직무 맞춤형 동적 페르소나 부여를 요구조건으로 설정했으나, LLM이 사용자 답변의 정확도·논리성을 실시간 채점하고 그 점수에 연동해 면접관의 태도를 긍정형과 부정형 사이에서 실시간 전환하는 답변 정확도 기반 가변 페르소나 제어로 재정의하였다. 이에 따라 과제 목표 자체도 가변적 AI 페르소나의 실시간 비언어적 피드백이 면접 훈련 효과에 미치는 효용성을 정량적으로 입증하는 방향으로 구체화되었다.

가변 페르소나는 두 축으로 정의한다. 먼저, 태도 축을 연속 강도로 정의하여 고정된 두 상태가 아닌 하나의 연속 스케일로 두고, 답변 정확도 채점 결과를 이 스케일에 매핑한다. 이로써 동일한 상태라도 강도에 따라 세분화될 수 있도록 구현하였다. 다음으로, 발화 상태 축은 경청, 말하기, 끼어들기의 세 가지로 정의한다. 사용자의 발화를 듣는 경청, 면접관이 질문하거나 발언하는 말하기, 그리고 사용자의 발화 도중 면접관이 말을 끊고 들어오는 끼어들기이다. 이 중 끼어들기는 면접관 에이전트가 사용자의 답변 도중 능동적으로 개입하는 기능으로, 본 시스템의 핵심 기능이다. 답변 정확도 채점과 행동 신호를 근거로 개입 시점을 판단하며, 답변이 지나치게 길어지거나 명백한 오류가 감지되는 등의 상황에서 개입한다.

1.2 TTS 모델 교체

착수보고서에서는 TTS 모델로 XTTS-v2를 로컬 서버에 배치하는 것을 계획하였으나, 구현 과정에서 FishSpeech v1.5로 교체하였다. 그 이유는 테스트 과정에서 XTTS-v2가 지연 시간과 소모 리소스 측면에서는 준수했지만 모델 학습에 사용된 한국어 음성 데이터가 적었던 탓인지 합성된 한국어 음성의 품질이 매우 낮았기 때문이다.

FishSpeech v1.5 역시 XTTS-v2와 유사하게 10초 내외의 레퍼런스 음성만으로 zero-shot 보이스 클로닝이 가능한 모델이지만, 20,000 시간 가량의 한국어 음성 데이터를 학습했기 때문에 테스트 결과 훨씬 개선된 음성 품질을 보였다.

1.3 VR에서 PC 기반으로의 전환

현재 구현 범위에서 HMD의 필요성이 크지 않고, 몰입형 HMD보다 대형 화면 앞에서 가상 휴먼과 상호작용하며 사용자의 수치적 행동 측정에 집중하는 것으로 요구사항을 변경하였다. 이에 따라 HMD 기반 VR을 제외하고 PC 기반으로 플랫폼을 전환하여 기존의 HMD 기반 행동 데이터를 수집하는 방식에서 웹캠 기반으로 행동 데이터를 수집하도록 변경하였다.

2. 설계 상세화 및 변경 내역

2.1 개발 언어 및 개발 도구

2.1.1 개발 언어

- C#: Unity 클라이언트 개발
- Python: 음성 처리 서버 및 AI/백엔드 서버 개발

2.1.2 개발 도구

- Unity: VR 면접 공간 및 클라이언트 구현
- Character Creator 5, ActorCore: 면접관 캐릭터 제작
- uLipSync: 면접관 캐릭터의 립싱크 구현
- FastAPI: 유니티와의 연결에 필요한 AI 백엔드 서버 및 STT/TTS 워커의 API(인터페이스) 구현
- Uvicorn: FastAPI로 정의한 백엔드 서버 및 워커를 실제로 구동하는 비동기 웹 서버
- WebSocket: 유니티와 워커 간의 저지연 네트워크 연결 프로토콜
- OpenAI gpt-4o-mini: AI/백엔드 서버에서 자기소개 정보 추출, 답변 채점, 페르소나별 꼬리 질문 및 비언어 메타데이터 생성을 담당하는 LLM
- Whisper large-v3-turbo (CTranslate2): CTranslate2 추론 엔진을 적용하여 GPU VRAM 점유율을 낮춘 STT 워커에 사용된 음성 인식 모델
- FishSpeech v1.5: TTS 워커에 사용되는 LLM 답변 텍스트를 음성으로 합성하기 위한 모델
- Silero VAD: 정적인 임계값 사용 시 발생하는 발화 종료 판단 지연을 줄이기 위해 실시간 음성을 동적으로 분석함으로써 지연을 최소화하는 모델
- Docker Desktop: 컨테이너화
- Dev Containers: docker container 내 vs code를 이용한 효율적인 개발 환경
- GitHub: 버전 관리 시스템

2.2 시스템 구조

2.2.1 사용자 인터페이스(프론트엔드)

- 고품질 3D 면접실 공간 렌더링

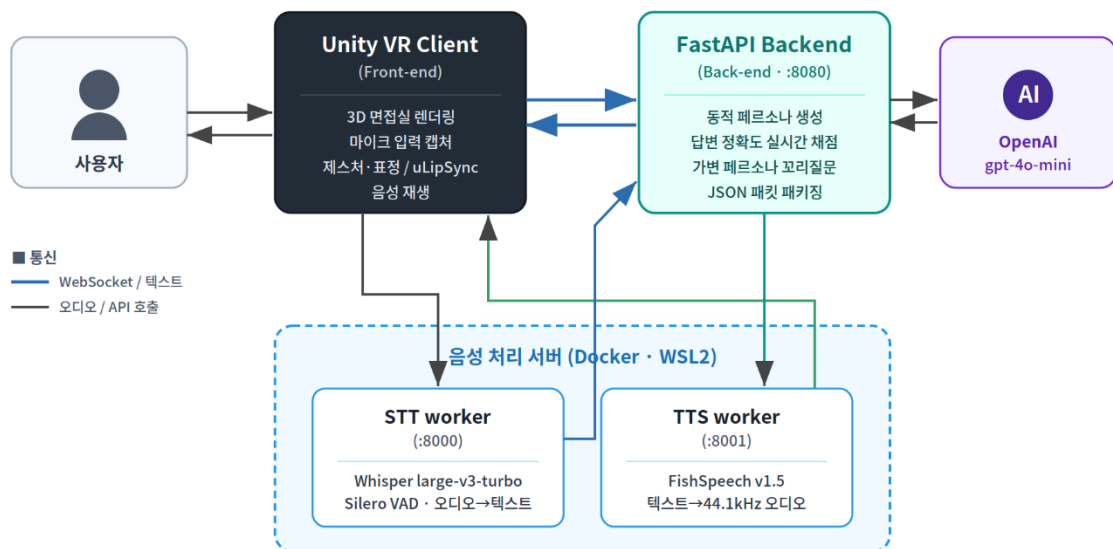
- 사용자 마이크 입력 캡처 및 서버 전송
- 백엔드 JSON 패킷 파싱 후 가변 페르소나에 따른 제스처, 표정 애니메이션 트리거
- uLipSync를 통한 실시간 음소 식별 및 입 모양 동기화

2.2.2 서버 측 애플리케이션(백엔드)

- STT worker (8000): Whisper 모델로 오디오 → 텍스트 변환
- TTS worker (8001): FishSpeech v1.5로 텍스트 → 44.1kHz 오디오 스트리밍 합성
- AI/백엔드 서버 (8080): FastAPI 기반, gpt-4o-mini로 페르소나 생성·채점·꼬리질문 생성
- Docker 컨테이너로 각 모델의 CUDA·라이브러리 환경 분리 구동
- WebSocket 전이중 통신으로 대사·제스처·채점 결과 패킷 전송

2.3 전체 구조도

클라이언트 - Unity - WebSocket - FastAPI(백엔드 8080) 구조를 중심으로, 백엔드가 STT worker(8000), TTS worker(8001), OpenAI gpt-4o-mini API와 연동된다



2.4 시스템 흐름

면접 진행 시나리오

1. 환경 초기화: HMD 착용 시 앓은 상태 기준의 3D 면접장이 구성
2. 자기소개: 면접관이 타겟 기업과 지원 직무를 포함한 자기소개를 요구하고, 그 답변을 바탕으로 직무 맞춤형 AI 면접관 페르소나가 동적으로 활성화
3. 기술 질문 1: 자기소개 종료를 파악한 뒤, 직무 핵심 역량을 검증하는 기술 질문을 도출
4. 1차 꼬리 질문: 답변의 정확도와 논리성을 실시간으로 채점하여, 정확도가 높으면 긍정 페르소나로, 낮거나 모호하면 부정 페르소나로 전환
5. 2차 꼬리 질문: 재답변을 문맥 추론하여 STAR 기법의 행동과 결과 유무를 재채점하며, 정확도가 계속 낮으면 부정 페르소나가 강화
6. 인성 질문: 협업과 갈등 해결 능력을 검증하는 인성 세션으로 분위기를 완화하며 전환
7. 마무리: 마지막 발언과 질문 기회를 제공한 뒤 정중한 인사로 세션을 종료하고, 누적 대화 및 멀티모달 반응 데이터를 종합한 피드백을 UI로 시각화

실시간 데이터 흐름

1. 발화: 면접관 질문 종료 후 사용자가 음성으로 답변
2. 데이터 전송: Unity 클라이언트가 오디오 버퍼는 음성 처리 서버로, 멀티모달 데이터는 AI 서버로 WebSocket을 통해 병렬 전송(구현 예정)
3. 음성 처리: 음성 처리 서버가 Whisper로 오디오를 텍스트로 변환하고, 분석한 특성 값들을 AI 서버로 전달
4. AI 평가 및 생성: AI 서버가 답변 정확도와 논리성을 채점하여 페르소나 톤을 확정하고, Function Calling으로 비언어적 피드백 코드를 선택한 뒤 꼬리 질문을 생성하여 [대사 텍스트 + 제스처 메타데이터 + 정확도 점수]를 단일 JSON으로 패키징
5. TTS 및 출력: 텍스트를 오디오로 합성해 Unity로 전송하고, Unity 클라이언트는 오디오 재생과 동시에 uLipSync로 입 모양을 동기화하며 JSON에 매핑된 비언어적 피드백을 출력

2.5 변경 내역

- TTS 모델: XTTS-v2 → FishSpeech v1.5로 교체
- 면접관 페르소나: 직무 맞춤형 정적 페르소나를 답변 정확도 기반 가변 페르소나로 재정의
- STT: Whisper large-v3-turbo에 FLOAT16 반정밀도 적용으로 구체화
- 침묵 감지: 추상적 요구사항 → Unity에서 RMS 기반 정적 임계값 판정 + Silero VAD 채택으로 확정
- 면접 진행 시퀀스 확정: 자기소개 → 기술1 → 꼬리1 → 꼬리2 → 인성 → 마무리
- Function Calling 확장: 정보 추출 + 채점 점수와 Gesture_ID, Expression_ID를 단일 JSON 스키마로 바인딩
- 플랫폼: HMD 기반 VR에서 PC 기반으로 전환하고 웹캠 기반 행동 데이터 수집을 도입
- 가변 페르소나 강화: 태도 축을 연속 강도로 확장하고 발화 상태 축에 개입 상태를 추가
- 면접관 능동 개입 기능 도입

3. 갱신된 과제 추진 계획

개발 구분	세부항목	5월(완료)	6월(완료)	7월(진행중)	8월	9월
기획 및 설계	요구사항 확정 및 시스템 아키텍처 상세 설계					
	면접관 페르소나 및 면접 시나리오 설계					
환경 및 모듈 개발	유니티 면접장 공간 구축 및 3D 에셋 최적화					
	웹캠 기반 멀티모달 행동 데이터 수집 모듈 개발					
	STT, TTS 서버 구축					
	LLM 프롬프트 엔지니어링 및 컨텍스트 관리					

통합 및 기능 구현	WebSocket 기반 유니티-서버 실시간 연동					
	실시간 링크 및 연속형 페르소나 행동 패턴 구현					
	다각적 분석 기반 자동 피드백 리포트 시스템					
	면접관 실시간 개입 기능 구현					
테스트 및 고도화	지연 시간 최적화					
최종 마무리	종합 시뮬레이션 테스트 및 최종 보고서 작성					

4. 구성원별 진척도

유재명

1. Unity 기반 3D 면접장 공간 구축(완료)
2. 면접관 Virtual Human 모델 제작(완료)
3. Python 비동기 FastAPI 서버 구축(완료)
4. Unity와 서버 간 WebSocket 전이중 통신 채널 구현(완료)
5. 연속형 가변 페르소나에 따른 면접관 표정 및 제스처 애니메이션 바인딩(진행 중)
6. uLipSync 기반 실시간 립싱크 동기화(완료)
7. 웹캠 기반 멀티모달 행동 데이터 수집(미구현)
8. 면접결과 화면 UI 제작(완료)
9. 면접관 실시간 개입 기능 구현(미구현)

강저릭 어요습드

1. Function Calling 기반 자기소개 정보 추출 모듈 구현 – 기업명, 직무, 경력 수준, 기술 스택, 강점을 구조화된 스키마로 추출(완료)
2. 답변 정확도 기반 가변 페르소나 System Prompt 동적 생성 로직 구현 (완료)
3. 면접 단계(자기소개→기술1→꼬리1→꼬리2→인성→마무리)별 지시문 및 턴 생성 로직 구현 (완료)
4. 대사, 채점 점수, 제스처/표정 ID를 단일 JSON으로 강제 출력하는 LLM 응답 계약 구현 (완료)
5. 페르소나 허용 세트를 벗어난 제스처/표정 ID 보정 및 JSON 파싱 실패 시 재시도, 폴백 처리 구현(완료)
6. 면접 종료 후 강점, 개선점, 총평을 생성하는 종합 피드백 모듈 구현(완료)
7. STT 음성 특징 데이터(발화 시간, 침묵 횟수)를 반영한 피드백 고도화(진행 중)

강동요

1. Docker 컨테이너 환경의 GPU 가속 기반 비동기 STT, TTS 워커 구축 (FastAPI, Whisper, FishSpeech 1.5) (완료)
2. STT 결과 신뢰도가 낮을 시 사용자가 결과를 보고 잘못된 부분에 관하여 재발화를 가능하게 하는 교정 UI 구현 (완료)
3. TTS 워커로부터 수신된 오디오 PCM 데이터를 실시간으로 버퍼에 적재하여 재생하는 스트리밍 오디오 플레이어 구현 (완료)
4. 사용자의 발화 시간, 침묵 횟수, 평균 음량 등의 음성적 특징 데이터 추출 및 서버 송신 기능 구현 (진행 중)
5. 사용자의 행동적 특징 데이터 추출 및 서버 송신 (미구현)

5. 연구 차별점 및 평가 계획

5.1 기존 연구와의 차별점

기존 가상 면접 시뮬레이터는 대부분 턴제로 진행되며, 최근에는 LLM을 활용해 이력서, 직무, 대화 이력을 반영한 꼬리 질문을 생성하는 수준까지 발전하였다. 한편 대화 에이전트의 개입 연구는 대부분 사용자가 에이전트를 끄는 방향에 집중되어 있으며, 에이전트가 사용자를 끄는 기능은 오히려 의도적으로 배제되어 왔다.

본 프로젝트는 실시간 답변 채점과 수치적 행동 신호를 근거로 면접관이 능동적으로 개입하여 압박 강도를 제어하는 교육용 가상 면접관을 차별점으로 제시한다.

5.2 평가 방법

차별점의 효용을 검증하기 위해 세 가지 조건을 비교한다. 첫째는 기존 가상 면접 시뮬레이터 방식인 정적 턴제 면접관, 둘째는 가변 페르소나, 셋째는 가변 페르소나와 능동 개입을 결합한 조건이다. 세 조건은 별도의 시스템이 아니라 동일 시스템에서 페르소나 전이와 개입 기능을 켜고 끄는 방식으로 구성하며, 이를 통해 변수 하나만 바꾸어 비교하는 실험적 엄밀성을 확보한다.

평가는 두 가지 지표로 수행한다. 첫째, 행동 지표는 웹캠과 마이크로 발화 시간, 침묵 횟수와 길이, 필러 빈도, 시선 안정도, 자세 변화 등을 객관적이고 자동으로 측정한다. 압박이 실제로 유도되었다면 개입 조건에서 이 수치들이 유의하게 달라질 것으로 기대한다. 둘째, 수행 지표는 동일 사용자가 세션을 반복할 때 답변의 구조성, 논리성, STAR 기법 준수도가 개선되는 정도를 조건별로 비교한다. 실험은 참가자 내 설계로 진행하되 순서 효과를 통제하기 위해 조건 제시 순서를 무작위화한다.

6. 보고 시점까지의 과제 수행 내용 및 중간 결과

1. Unity 기반 3D 면접장 공간



2. 면접관 Virtual Human 모델 제작



3. 음성 대화 파이프라인

3.1 유니티



3.2 백엔드 서버

```
[INFO:      ('127.0.0.1', 56990) - "WebSocket /ws/control" [accepted]
[INFO:      connection open
[INFO:      ('127.0.0.1', 56992) - "WebSocket /ws/tts?session_id=default" [accepted]
[/ws/tts] STT 워커 연결됨 - Session ID: default
[INFO:      connection open
[default] Persistent connection to TTS Worker established.
[STT→백엔드 수신 (default)] 저는 네이버의 백엔드로 지원한 유재명입니다.
[백엔드→TTS 대사] 유재명님, Spring과 Java를 3년 동안 사용하셨다고 하셨는데, 그 과정에서 가장 도전적이었던 프로젝트나
문제는 무엇이었고, 어떻게 해결하셨는지 구체적으로 말씀해 주실 수 있나요? (stage=TECH_Q1, persona=NEUTRAL, score=-1)
[INFO:      connection closed
[/ws/tts] [default] TTS 릴레이 실패: sent 1000 (OK); then received 1000 (OK)
[default] Persistent connection to TTS Worker closed.
[/ws/tts] STT 워커 연결 종료 - Session ID: default
[INFO:      connection closed
```

3.3 STT worker

```
INFO: 127.0.0.1:34312 - "WebSocket /ws/interview?session_id=default" [accepted]
WebSocket Connected (PCM Stream Mode) - Session ID: default
INFO: connection open
[default] TTS Worker connection offline. Connecting...
[default] Persistent connection to TTS Worker established.
VAD detected 3 consecutive silences. Requesting end...
Raw Unity JSON: {"type": "websocket.receive", "text": '{"type": "utterance_end", "session_id": "default", "features": {"speakingTime": 0.00, "pauseCount": 1, "averageVolume": 0.2067118}}'}
Utterance End received. Transcribing 9605486 bytes...
STT Success: '저는 네이버의 백엔드로 지원한 유재명입니다.' (Confidence: 0.87)
Requesting TTS for: 저는 네이버의 백엔드로 지원한 유재명입니다.
Sent text to TTS Worker: 저는 네이버의 백엔드로 지원한 유재명입니다.
Critical Error: Cannot call "receive" once a disconnect message has been received.
INFO: connection closed
Persistent connection to TTS Worker closed.
```

3.4 TTS worker

```
INFO: 127.0.0.1:5256 - "websocket /ws/tts" [accepted]
WS: Persistent client connection accepted.
INFO: connection open
TTS: Synthesizing: 안녕하세요.
2025-07-04 11:19:31.611 INFO | fish_speech.models.text2semantic.inference:generate_log:788 - Encoded text: 안녕하세요.
2025-07-04 11:19:31.611 INFO | fish_speech.models.text2semantic.inference:generate_log:806 - Generating sentence 1/1 of sample 1/1
13/1023 [00:00:00:14, 94.321t/s]
2025-07-04 11:19:31.821 INFO | fish_speech.models.text2semantic.inference:generate_log:860 - Generated 15 tokens in 0.21 seconds, 71.68 tokens/sec
2025-07-04 11:19:31.821 INFO | fish_speech.models.text2semantic.inference:generate_log:863 - Bandwidth achieved: 45.73 GB/s
2025-07-04 11:19:31.821 INFO | fish_speech.models.text2semantic.inference:generate_log:868 - GPU Memory used: 2.03 GB
TTS: Synthesizing: 면접에 합격해 주셔서 감사드립니다.
2025-07-04 11:19:31.932 INFO | fish_speech.models.text2semantic.inference:generate_log:788 - Encoded text: 면접에 합격해 주셔서 감사드립니다.
2025-07-04 11:19:31.933 INFO | fish_speech.models.text2semantic.inference:generate_log:806 - Generating sentence 1/1 of sample 1/1
41/1023 [00:00:00:14, 68.411t/s]
2025-07-04 11:19:32.088 INFO | fish_speech.models.text2semantic.inference:generate_log:860 - Generated 43 tokens in 0.68 seconds, 63.69 tokens/sec
2025-07-04 11:19:32.088 INFO | fish_speech.models.text2semantic.inference:generate_log:863 - Bandwidth achieved: 40.63 GB/s
2025-07-04 11:19:32.088 INFO | fish_speech.models.text2semantic.inference:generate_log:868 - GPU Memory used: 2.03 GB
TTS: Synthesizing: 면접.
2025-07-04 11:19:32.714 INFO | fish_speech.models.text2semantic.inference:generate_log:788 - Encoded text: 면접.
2025-07-04 11:19:32.715 INFO | fish_speech.models.text2semantic.inference:generate_log:806 - Generating sentence 1/1 of sample 1/1
8/1023 [00:00:00:13, 73.491t/s]
2025-07-04 11:19:32.912 INFO | fish_speech.models.text2semantic.inference:generate_log:860 - Generated 10 tokens in 0.20 seconds, 50.65 tokens/sec
2025-07-04 11:19:32.912 INFO | fish_speech.models.text2semantic.inference:generate_log:863 - Bandwidth achieved: 32.31 GB/s
2025-07-04 11:19:32.912 INFO | fish_speech.models.text2semantic.inference:generate_log:868 - GPU Memory used: 2.03 GB
TTS: Synthesizing: 본인 소개와 함께 지원하신 기업과 직무에 대한 간단한 말씀 부탁드립니다.
2025-07-04 11:19:32.999 INFO | fish_speech.models.text2semantic.inference:generate_log:788 - Encoded text: 본인 소개와 함께 지원하신 기업과 직무에 대한 간단한 말씀 부탁드립니다.
2025-07-04 11:19:32.999 INFO | fish_speech.models.text2semantic.inference:generate_log:806 - Generating sentence 1/1 of sample 1/1
183/1023 [00:02:00:09, 88.821t/s]
2025-07-04 11:19:35.148 INFO | fish_speech.models.text2semantic.inference:generate_log:860 - Generated 185 tokens in 2.15 seconds, 86.11 tokens/sec
2025-07-04 11:19:35.148 INFO | fish_speech.models.text2semantic.inference:generate_log:863 - Bandwidth achieved: 54.93 GB/s
2025-07-04 11:19:35.148 INFO | fish_speech.models.text2semantic.inference:generate_log:868 - GPU Memory used: 2.03 GB
TTS: Synthesizing: 유재현.
2025-07-04 11:19:35.159 INFO | fish_speech.models.text2semantic.inference:generate_log:788 - Encoded text: 유재현.
2025-07-04 11:19:35.159 INFO | fish_speech.models.text2semantic.inference:generate_log:806 - Generating sentence 1/1 of sample 1/1
16/1023 [00:00:00:12, 81.581t/s]
2025-07-04 11:19:35.462 INFO | fish_speech.models.text2semantic.inference:generate_log:860 - Generated 18 tokens in 0.30 seconds, 59.45 tokens/sec
2025-07-04 11:19:35.462 INFO | fish_speech.models.text2semantic.inference:generate_log:863 - Bandwidth achieved: 37.93 GB/s
2025-07-04 11:19:35.463 INFO | fish_speech.models.text2semantic.inference:generate_log:868 - GPU Memory used: 2.15 GB
TTS: Synthesizing: Spring과 Java를 3년 동안 사용해오셨다고 하셨습니다.
2025-07-04 11:19:35.526 INFO | fish_speech.models.text2semantic.inference:generate_log:788 - Encoded text: Spring과 Java를 3년 동안 사용해오셨다고 하셨습니다.
2025-07-04 11:19:35.527 INFO | fish_speech.models.text2semantic.inference:generate_log:806 - Generating sentence 1/1 of sample 1/1
71/1023 [00:00:00:10, 89.251t/s]
2025-07-04 11:19:36.416 INFO | fish_speech.models.text2semantic.inference:generate_log:860 - Generated 73 tokens in 0.89 seconds, 82.15 tokens/sec
2025-07-04 11:19:36.416 INFO | fish_speech.models.text2semantic.inference:generate_log:863 - Bandwidth achieved: 52.48 GB/s
2025-07-04 11:19:36.416 INFO | fish_speech.models.text2semantic.inference:generate_log:868 - GPU Memory used: 2.15 GB
TTS: Synthesizing: 그 과정에서 가장 도전적이었던 프로젝트나 문제는 무엇이었고,
2025-07-04 11:19:36.541 INFO | fish_speech.models.text2semantic.inference:generate_log:788 - Encoded text: 그 과정에서 가장 도전적이었던 프로젝트나 문제는 무엇이었고,
2025-07-04 11:19:36.544 INFO | fish_speech.models.text2semantic.inference:generate_log:806 - Generating sentence 1/1 of sample 1/1
81/1023 [00:00:00:10, 88.801t/s]
2025-07-04 11:19:37.553 INFO | fish_speech.models.text2semantic.inference:generate_log:860 - Generated 83 tokens in 1.01 seconds, 82.27 tokens/sec
2025-07-04 11:19:37.553 INFO | fish_speech.models.text2semantic.inference:generate_log:863 - Bandwidth achieved: 52.48 GB/s
2025-07-04 11:19:37.553 INFO | fish_speech.models.text2semantic.inference:generate_log:868 - GPU Memory used: 2.15 GB
TTS: Synthesizing: 어떻게 해결하셨는지 구체적으로 말씀해 주실 수 있나요?
2025-07-04 11:19:37.676 INFO | fish_speech.models.text2semantic.inference:generate_log:788 - Encoded text: 어떻게 해결하셨는지 구체적으로 말씀해 주실 수 있나요?
2025-07-04 11:19:37.676 INFO | fish_speech.models.text2semantic.inference:generate_log:806 - Generating sentence 1/1 of sample 1/1
52/1023 [00:00:00:09, 98.631t/s]
WS: connection closed
INFO: Client disconnected
142/1023 [00:01:00:09, 91.961t/s]
```

4. 시를 이용한 면접관 페르소나 테스트

```
(venv) PS C:\dev\vroom-llm> python example.py

자기소개를 입력해주세요.
안녕하세요. 저는 네이버 백엔드 개발자 직무에 지원한 홍길동입니다.
컴퓨터공학을 전공했고 신입으로 지원합니다.
주로 Python과 Spring Boot를 사용했으며, 작년에 RAG 기반 챗봇 프로젝트와 대규모 트래픽 처리 백엔드 시스템을 팀으로 개발한 경험이 있습니다.
문제를 끝까지 파고드는 끈기와 협업 능력이 강점입니다.

{
  "company_name": "네이버",
  "job_role": "백엔드 개발자",
  "experience_level": "신입",
  "mentioned_skills": [
    "Python",
    "Spring Boot"
  ],
  "key_strengths": [
    "끈기",
    "협업 능력"
  ]
}

동적 생성된 페르소나 System Prompt:
당신은 네이버의 백엔드 개발자 채용 면접관입니다.

지원자 정보:
- 경력 수준: 신입
- 보유 기술: Python, Spring Boot
- 강조한 강점: 끈기, 협업 능력

면접관 가이드라인:
- 지원자가 언급한 기술과 경험에 대해 깊이 있는 질문을 합니다
- 지원자의 *직접 답변 내용을 구체적으로 인용*하며 꼬리 질문을 만듭니다
- STAR 기법(상황-과제-행동-결과) 중 부족하거나 모호한 부분이 있으면 꼭 물어봅니다
- 신입 수준에 적합한 난이도로 조정합니다
- 전문적이지만 위협적이지 않은 톤을 유지합니다
- 질문은 한 번에 하나만, 2-3 문장 이내로 짧게 합니다
- 한 주제에서 충분히 답변을 받았다 싶으면 자연스럽게 다른 주제로 넘어갑니다
```

면접관 (turn 1): 홍길동님, 말씀하신 RAG 기반 챗봇 프로젝트에 대해 좀 더 구체적으로 말씀해 주실 수 있나요? 프로젝트에서 어떤 상황에서 어떤 과제를 맡고, 그에 대해 어떤 행동을 취했는지, 그리고 결과는 어떻게 되었는지 설명해 주세요.

질문에 답변을 입력해주세요 :

제가 다니던 대학교 외국인 학생들을 위한 한국 유학 생활 적응을 목적으로 챗봇을 만들었습니다. 필요한 데이터들을 학교 홈페이지에서 불러와서 테이블이나 그래프 형태로 되어있는 데이터들을 파싱한 다음 RAG를 사용해 챗봇을 만들었습니다/

면접관 (turn 2): 챗봇 제작에서 데이터 파싱을 하셨다고 하셨는데요, 구체적으로 어떤 데이터 형식을 사용했고, 파싱 과정에서 어떤 도구나 방법론을 활용하셨는지 설명해 주실 수 있나요? 파싱 중에 맞닥뜨린 어려움도 함께 공유해 주시면 좋겠습니다.

질문에 답변을 입력해주세요 :

학교 홈페이지에서 가져온 데이터들이다 보니까 거의 테이블 형태로 되어 있는 데이터가 많았습니다. 그 데이터를 llm api key를 활용해 파싱해서 api key를 활용해 챗봇을 만들었기 때문에 별개의 어려움이 없고 챗봇이 잘 작동했던 것 같습니다.

면접관 (turn 3): 잘 작동했다고 하셨는데, 그러면 챗봇의 성능이나 사용자 반응에 대한 피드백은 어떻게 받으셨나요? 사용자의 피드백이나 데이터 분석 결과를 통해 어떤 개선 혹은 발전을 시도해본 경험이 있는지 알고 싶습니다.

질문에 답변을 입력해주세요 :

종료

면접 세션을 종료합니다. 수고하셨습니다.

총 3개 질문, 누적 토큰: 1949

대략 비용 추정: \$0.0006

o (venv) PS C:\dev\vroom-llm> █